Studio Nx

Backend(FastAPI/Python) 49 — purge_withdrawn_accounts()

2026-10-12

Backend(FastAPI/Python) 49 — `purge_withdrawn_accounts()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-09 비밀번호 변경·탈퇴: 탈퇴 3일 뒤 개인 정보 정리 (PRD 01 FR-108, 개인정보 처리방침 §4)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	48 `DELETE /users/me` · 50 `InquiryCreateRequest` (P1-12)

1. 하는 일

하루에 한 번 실행하는 **정리 작업**입니다. 탈퇴한 지 3일이 지난 계정에서 개인 정보를 비웁니다. 처리방침(privacy.html §4)에 적은 "탈퇴하면 3일 동안 보관한 뒤 비운다"는 약속을 실제로 지키는 곳입니다.

칸	정리 전	정리 후
`email`	`old.gone@example.com`	`deleted+85@withdrawn.invalid`
`nickname`	`old_gone`	`deleted_user_0000000085`
`password_hash`	`$2b$12$…`	`!` (어떤 비밀번호와도 맞지 않습니다)
`last_login_at`	마지막 로그인 시각	`None`
`profile_image_path`와 사진 파일	(남아 있다면)	`None`, 파일 삭제
`status`, `deleted_at`, 행 자체, 프로젝트		**그대로 둡니다**

계정	결과
탈퇴한 지 4일, 정확히 3일	정리합니다
탈퇴한 지 2일	그대로 둡니다. 3일 동안은 재가입을 막는 데 이메일과 닉네임을 쓰기 때문입니다
활동 중인 회원	그대로 둡니다
이미 모두 비운 계정	건너뜁니다 (세지도 않습니다)
다른 사람이 그 이메일로 가입하면서 이메일과 닉네임만 비켜 둔 계정	남아 있는 비밀번호 해시 등을 마저 비웁니다

돌려주는 값은 이번에 비운 계정의 수입니다.

2. 언제 쓰나

실행	언제
`python -m app.workers.retention`의 `main()`	하루에 한 번 실행합니다 (운영체제의 예약 작업이며, 배포할 때 정합니다). **`main()`은 P1-12에서** 문의 정리(`purge_old_inquiries`)와 함께 부르도록 그때 구현합니다

탈퇴 회원의 시간표

시점	상태	처리하는 곳
탈퇴하는 순간	`DELETED`로 바뀌고, 사진이 지워지며, 로그인과 재발급이 막힙니다	46, 48
3일까지	같은 이메일이나 닉네임으로 가입할 수 없습니다 (409 `REJOIN_TOO_SOON`)	P1-04
3일 뒤 누군가 그 값을 쓰려고 함	이메일과 닉네임만 바로 비켜 둡니다	P1-04 `release_withdrawn`
3일 뒤 첫 정리 작업	**나머지까지 모두 비웁니다**	**49**

추적 방지 (2026-10-07 결정 ②)

비켜 둔 값에는 회원 번호만 남고, 원래의 이메일, 닉네임, 비밀번호 해시, 마지막 로그인 시각, 사진은 남지 않습니다. 결정 ①(다른 사람에게 보이는 곳에는 'Deleted user'로 표시)과 결정 ③(회원 번호로 조회하는 공개 API를 두지 않음, P1-08 사진 주소 포함)과 함께 적용합니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`datetime` 줄과 `Session` 줄을 뺀 나머지는 모두 이번에 추가했습니다.

상수

```
# 다 비운 계정의 비밀번호 칸 값 — bcrypt 해시 모양이 아니라 verify_password 가 늘 False
# (칸이 NOT NULL 이라 빈칸 대신)
WIPED_PASSWORD_HASH = "!"
```

함수

```
def purge_withdrawn_accounts(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=REJOIN_WAIT_DAYS)
    users = db.scalars(
        select(User).where(
            User.status == UserStatus.DELETED,
            User.deleted_at <= cutoff,
            or_(
                ~User.email.startswith("deleted+", autoescape=True),
                ~User.nickname.startswith("deleted_user_", autoescape=True),
```

```
                User.password_hash != WIPED_PASSWORD_HASH,
                User.last_login_at.is_not(None),
                User.profile_image_path.is_not(None),
            ),
        )
    ).all()

    photos = []
    for user in users:
        release_withdrawn(db, user)
        if user.profile_image_path:
            photos.append(user.profile_image_path)
        user.password_hash = WIPED_PASSWORD_HASH
        user.last_login_at = None
        user.profile_image_path = None
    db.commit()

    storage_dir = get_settings().storage_dir
    for relative in photos:
        (storage_dir / relative).unlink(missing_ok=True)
    return len(users)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import or_, select`	`or_(A, B, …)`는 SQL의 `A OR B OR …`입니다
`from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn`	3일이라는 값과 비켜 두는 함수를 가입 쪽(P1-04)과 **같은 것으로** 씁니다

식	뜻
`now.astimezone(UTC).replace(tzinfo=None)`	한국 시각처럼 시간대가 붙은 값을 UTC로 바꾸고 시간대 표시를 뗍니다. DB는 UTC로 저장하기 때문입니다
`User.email.startswith("deleted+", autoescape=True)`	SQL `LIKE 'deleted+%'`가 됩니다. `autoescape`는 값 안의 `_`와 `%`를 글자 그대로 다루게 합니다 (`LIKE`에서 `_`는 "아무 글자 하나"를 뜻하기 때문입니다)
`~조건`	NOT입니다
`.is_not(None)`	`IS NOT NULL`입니다
`db.scalars(…).all()`	조건에 맞는 `User`들을 목록으로 가져옵니다

4. 흐름

1. 기준 시각을 지금으로부터 3일 전(UTC)으로 정합니다.
2. 탈퇴했고, 기준 시각보다 오래되었으며, **아직 남아 있는 정보가 하나라도 있는** 계정을 찾습니다.
3. 계정마다 이메일과 닉네임을 비켜 두고, 사진 경로를 기억해 둔 뒤, 비밀번호, 마지막 로그인 시각, 사진 경로를 비웁니다.
4. 한 번에 저장합니다.
5. 저장한 뒤 사진 파일을 지웁니다 (이미 없으면 넘어갑니다).
6. 비운 계정 수를 돌려줍니다.

5. 왜 이렇게

선택	이유
"남아 있는 정보가 하나라도 있는" 행만 고름	매일 실행해도 같은 행을 다시 세지 않습니다. 또 가입 때 이메일과 닉네임만 비켜 둔 행도 빠뜨리지 않습니다. 템플릿은 "이메일이 `deleted+`로 시작하지 않는 행"만 골랐는데, 그러면 그런 행의 비밀번호 해시가 영원히 남게 됩니다
`password_hash = "!"`	이 칸은 `NOT NULL`이라 비울 수 없습니다. `"!"`는 bcrypt 해시 모양이 아니라서 어떤 비밀번호로도 로그인할 수 없습니다 (44번 시험에서 확인했습니다)
`release_withdrawn`을 그대로 씀	비켜 두는 값이 가입 쪽과 항상 같아집니다. 여러 번 불려도 결과가 같습니다
`deleted_at`과 행을 남김	프로젝트(owner_id)가 이 행을 가리키고 있습니다. 탈퇴 시각도 기록으로 필요합니다
사진 파일은 저장한 **뒤에** 지움	38, 39번과 같은 순서입니다. 저장이 실패하면 파일도 남습니다
`now`를 인자로 받음 (함수 안에서 시각을 재지 않음)	시험에서 시각을 정해서 넣을 수 있습니다 ("정확히 3일" 같은 경계값)
시간대가 붙은 `now`도 받음	실행하는 쪽이 한국 시각을 넘겨도 틀리지 않게 합니다

알아 둘 점 (색인): `status`와 `deleted_at` 조건으로 회원 표를 훑습니다. 하루에 한 번만 실행하므로 회원이 많아도 부담이 적습니다.

6. 시험 (`tests/test_retention.py` 새 파일, 8개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
탈퇴한 지 4일 (사진, 로그인 기록, 프로젝트 있음)	1을 돌려주고, 이메일과 닉네임이 비켜지며, 해시는 `!`, 마지막 로그인과 사진 경로는 `None`이 되고 파일이 지워집니다. 상태, 탈퇴 시각, 프로젝트는 그대로입니다
탈퇴한 지 2일, 활동 중인 회원	0을 돌려주고 그대로 둡니다
정확히 3일	정리합니다
두 번 실행	처음에는 1, 두 번째에는 0입니다
가입 때 이메일과 닉네임만 비켜 둔 행	1을 돌려주고, 해시와 로그인 기록도 비웁니다
사진 파일이 이미 없음	오류 없이 정리합니다
한국 시각 21:00 (= UTC 12:00)으로 실행	정확히 3일로 계산해서 정리합니다
정리한 뒤	옛 비밀번호로 로그인하면 401이고, 옛 이메일과 닉네임으로는 가입할 수 있습니다

전체 시험 352개가 통과했고, `ruff` 검사도 통과했습니다.